

Industrijski komunikacioni protokoli u EES

MREŽNI PROTOKOL 4 - IGRA POGAĐANJA

02. JANUAR 2026.

AUTORI:

PR22/2022 Anđela Torlaković

PR21/2022 Milica Jovanović

Sadržaj

1. Uvod.....	3
1.1. Opis problema	3
1.2. Ciljevi zadatka	3
2. Dizajn.....	3
2.1. Opis dizajna	3
2.2. Dizajn aplikacije	4
3. Strukture podataka	4
3.1. Razlozi biranja baš ove strukture	4
3.2. Opis i pojašnjenje strukture	5
4. Rezultati testiranja	5
4.1. Opis testova	5
4.2. Stress test 1 – masovno povezivanje klijenata	6
4.2.1. Opis testa	6
4.2.2. Rezultat testa	6
4.3. Stress test 2 – paralelno izvršavanje više pogađanja	8
4.3.1. Opis testa	8
4.3.2. Rezultat testa	8
5. Potencijalna unapređenja	10
5.1. Unapređenja	10
6. Zaključak	10
6.1. Zaključci doneti na osnovu rezultata testova	10

1. Uvod

1.1. Opis problema

Implementirati klijent server arhitekturu takvu da klijent šalje zahtev za povezivanje na server. Nakon toga, oni igraju igru pogađanja. Klijent šalje server zahtev da počne da pogađa. Za svakog registrovanog klijenta, server ima zaseban broj. Nakon ovoga klijent šalje svoje pretpostavke, a server odgovara sa informacijom da li je tačan broj veći, manji ili pogođen. Potrebno je da postoji propisani broj iteracija u okviru kojih klijent treba da pogodi da bi “pobedio”, ukoliko to ne postigne treba da dobije poruku o tome da je izgubio i vrednost tačnog broja.

1.2. Ciljevi zadatka

Ciljevi projekta su:

- Implementacija servera koji podržava veliki broj istovremenih TCP konekcija.
- Implementacija klijentske aplikacije sa menijem (connect/start/disconnect) i unosom pretpostavki.
- Dizajn binarnog komunikacionog protokola sa jasnim tipovima poruka.
- Paralelna obrada korisnika putem višestrukih niti na serveru.
- Implementacija mehanizama sinhronizacije (mutex, condition_variable) radi bezbedne konkurentne obrade.
- Testiranje sistema kroz stres testove (masovno povezivanje i paralelno slanje velikog broja poruka).

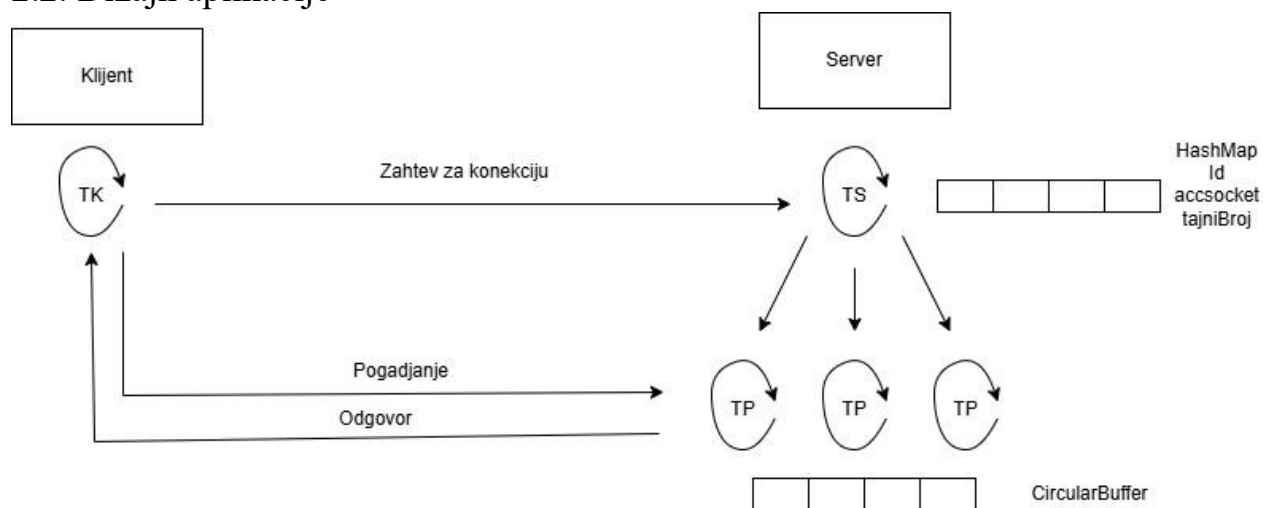
2. Dizajn

2.1. Opis dizajna

Serverska aplikacija igre pogađanja broja ima glavni thread koji je zadužen za prihvatanje dolaznih konekcija od klijenata. Nakon prihvatanja konekcije, za svakog klijenta se kreira poseban thread (client_handler) koji je zadužen za prijem i obradu svih zahteva tog klijenta tokom trajanja konekcije. Prilikom povezivanja, klijent šalje serveru svoje korisničko ime, nakon čega mu se dodeljuje jedinstveni identifikator i uspostavlja trajna komunikacija.

Svaki `client_handler` thread koristi kružni bafer za prijem poruka kako bi se omogućila ispravna obrada TCP poruka koje mogu stići parcijalno. Server za svakog klijenta čuva zasebno stanje igre (tajni broj i broj preostalih pokušaja), čime je omogućeno istovremeno igranje više klijenata bez međusobnog uticaja. Obrada zahteva i slanje odgovora vrši se unutar threada klijenta, dok je pristup zajedničkoj listi klijenata sinhronizovan pomoću mutex-a. Klijentska aplikacija koristi poseban thread za prijem poruka sa servera, što omogućava asinhrono primanje rezultata igre bez blokiranja korisničkog interfejsa.

2.2. Dizajn aplikacije



Slika 1. Dizajn aplikacije

3. Strukture podataka

3.1. Razlozi biranja baš ove strukture

Komunikacija se vrši putem binarnih poruka definisanih preko `MessageHeader` strukture. Ovakav format je izabran jer:

- omogućava brzu serijalizaciju,
- jednostavan je za parsiranje,
- pogodan je za industrijske protokole gde je bitna efikasnost.

Za upravljanje klijentima server koristi:

- sopstvenu povezanu listu `ClientList` (`Node` strukture),
- zaštitu kritičnih sekcija pomoću `std::mutex`.

Za obradu TCP stream-a i razdvajanje poruka korišćen je:

- CircularBuffer (kružni bafer) kako bi se podržalo parcijalno primanje podataka i korektno sastavljanje poruka.

3.2. Opis i pojašnjenje strukture

MessageHeader – metapodaci o poruci (client_id, request_type, payload_len)

- client_id – ID klijenta (dodeljuje server, klijent ga čuva kao my_id)
- request_type – tip poruke (CONNECT, START_GAME, GUESS, itd.)
- payload_len – dužina dodatnih podataka (username, guess broj, poruka servera...)

Client – podaci o klijentu, server čuva stanje po klijentu:

id, socket, username, game_active, secret_number, attempts_left, active

CircularBuffer - kružni buffer za prijem podataka. Koristi se za:

skladištenje dolaznih bajtova iz recv, izdvajanje kompletnih poruka (peek, pop), sprečavanje grešaka zbog TCP segmentacije (jer TCP ne garantuje da će cela poruka stići odjednom).

4. Rezultati testiranja

4.1. Opis testova

Testiranje obuhvata stres testove jer oni najrealnije simuliraju uslove industrijske komunikacije:

- veliki broj istovremenih konekcija,
- veliki broj poruka u kratkom vremenu,
- provera stabilnosti i performansi servera.

Implementirana su dva stres testa u posebnoj aplikaciji:

1. masovno povezivanje klijenata
2. paralelno slanje velikog broja pogađanja (message flood)

4.2. Stress test 1 – masovno povezivanje klijenata

4.2.1. Opis testa

Test simulira istovremeno povezivanje velikog broja klijenata. Svaki klijent:

- otvara TCP konekciju,
- šalje CONNECT poruku sa username-om,
- prima potvrdu servera (INFO: CONNECTED_OK),
- ostaje povezan dok se test ne prekine.

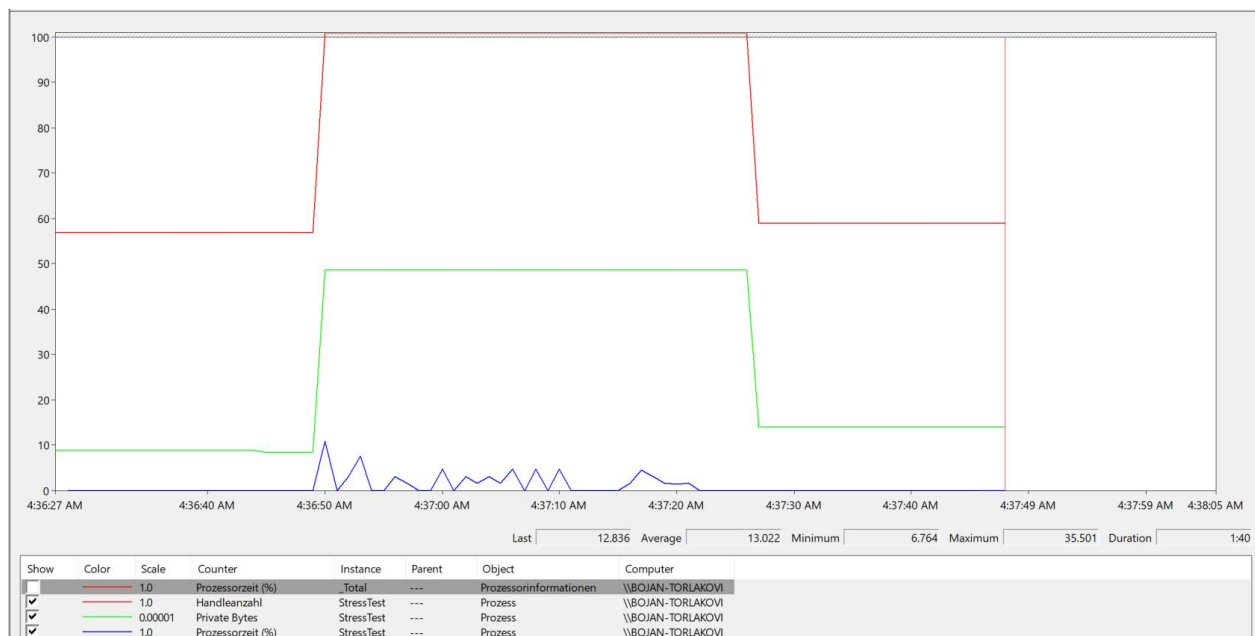
Cilj testa je da se proverí:

- stabilnost servera pri velikom broju konekcija,
- obrada registracije i dodela ID-jeva,
- ponašanje servera u uslovima paralelnog pristupa listi klijenata.

4.2.2. Rezultat testa

Test je izvršen sa **100 klijenata**. Mereno je:

- broj uspešno povezanih klijenata,
- broj klijenata koji su dobili ACK odgovor,
- broj neuspelih konekcija,
- ukupno vreme izvršavanja.



Slika 2. PerformanceMonitor Stress test 1

```

===== TEST 1 RESULTS =====
Attempted: 100
Connected: 100
ACK received: 100
Failed: 0
Time: 37114 ms

```

Slika 3. Rezultati Stress testa 1

	ID	Time	Allocations (Diff)	Heap Size (Diff)	Label
	1	4.66s	444 (n/a)	120.70 KB (n/a)	
	2	70.49s	2,190 (+1,746 ↑)	782.03 KB (+661.33 KB ↑)	
	3	106.63s	500 (-1,690 ↓)	143.81 KB (-638.22 KB ↓)	
	4	186.87s	498 (-2 ↓)	142.79 KB (-1.02 KB ↓)	
	5	190.90s	498 (+0)	142.79 KB (+0.00 KB)	

Slika 4. Memory snapshot za stress test 1

4.3. Stress test 2 – paralelno izvršavanje više pogađanja

4.3.1. Opis testa

U drugom stres testu simulira se paralelna aktivnost više klijenata koji istovremeno:

- šalju CONNECT,
- šalju START_GAME,
- zatim šalju veliki broj GUESS poruka.

U našoj implementaciji test je pokrenut sa:

- 50 klijenata
- 1000 poruka po klijentu

Ukupno poslato: 50,000 GUESS poruka.

Ovim testom se proverava:

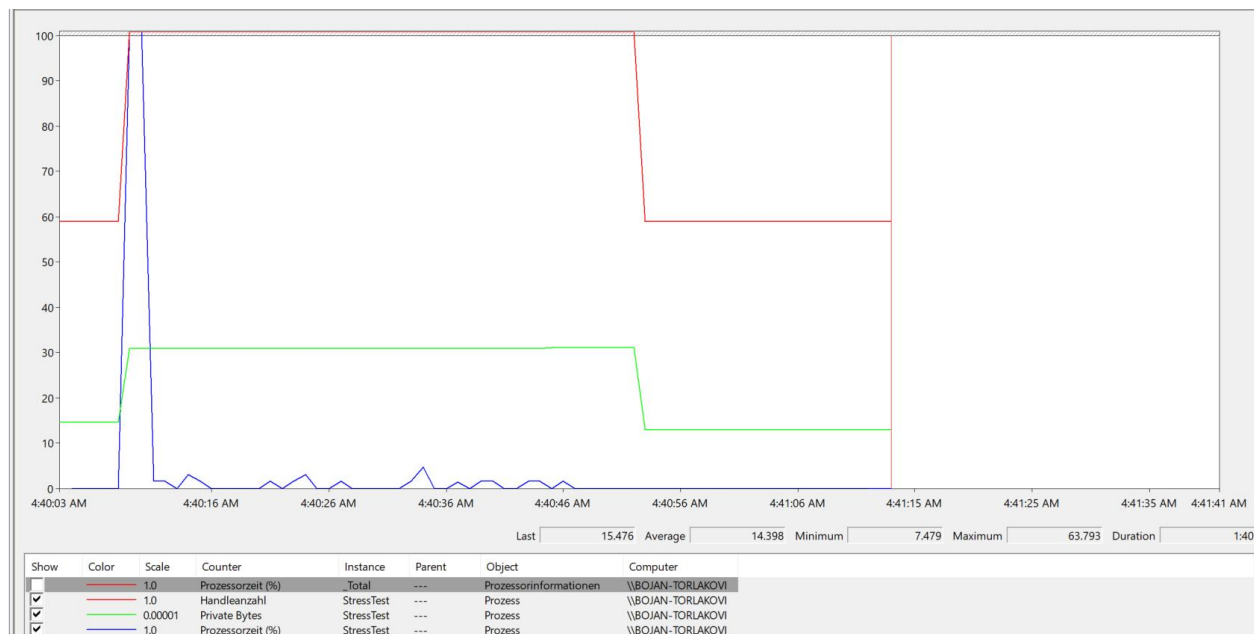
- throughput servera,
- stabilnost pri velikom broju poruka,
- korektno parsiranje poruka iz TCP stream-a (CircularBuffer).

4.3.2. Rezultat testa

Server je uspešno obradio veliki broj poruka bez rušenja i bez mešanja sesija između klijenata.

Prikazani su:

- broj poslatih poruka,
- vreme izvršavanja,
- snapshots memorije i CPU opterećenja.



Slika 5. PerformanceMonitor Stress test 2

```

===== TEST 2 RESULTS =====
Clients: 50
Messages sent: 50000
Time: 44028 ms
  
```

Slika 6. Rezultati Stress testa 2

5	190.90s	498	(+0)	142.79 KB	(+0.00 KB)
6	277.30s	1,348	(+850 ↑)	462.43 KB	(+319.64 KB ↑)
7	311.81s	511	(-837 ↓)	149.18 KB	(-313.24 KB ↓)
8	348.41s	511	(+0)	149.18 KB	(+0.00 KB)

Slika 7. Memory snapshot za stress test 2

5. Potencijalna unapređenja

5.1. Unapređenja

1. Thread pool umesto 1 nit po klijentu

Trenutno server za svakog klijenta kreira novu nit. Kod velikog broja korisnika to može postati neefikasno. Thread pool bi omogućio:

- manji overhead,
- bolje performanse,
- stabilniju obradu velikog broja konekcija.

2. Poboljšanje protokola (validacija/checksum)

Trenutno protokol sadrži header sa dužinom payload-a, ali nema checksum. Dodavanje checksum-a bi omogućilo:

- otkrivanje korupcije podataka,
- veću pouzdanost protokola.

3. Ograničenja i pravila igre

Moguće je dodati:

- fiksni broj pokušaja (umesto random),
- opcije težine (easy/medium/hard),
- ponovno pokretanje igre bez reconnect-a.

6. Zaključak

6.1. Zaključci doneti na osnovu rezultata testova

Na osnovu sprovedenih testova zaključuje se da je sistem stabilan i funkcionalan i u uslovima povećanog opterećenja. Server uspešno podržava:

- masovno povezivanje klijenata (100 simultanih konekcija),
- paralelnu obradu velikog broja poruka (50 klijenata $\times$ 1000 pogađanja).

Korišćenje višestrukih niti i kružnog bafera za parsiranje TCP stream-a omogućilo je pouzdanu obradu poruka i sprečilo gubitak podataka pri parcijalnom prijemu.

Dodatnim unapređenjima (thread pool, checksum, optimizacija zaključavanja) sistem bi mogao postići još veću skalabilnost i performanse.